Attempt any two questions.

Plagiarism is strictly discouraged. Any copied content may result in deduction of marks as per academic guidelines.

Submissions containing more than 30% AI-generated content will be awarded zero (0) marks.

Question 1.

A three-person indie studio is developing a 2D mobile action-platformer in Unity. The current prototype exhibits frequent frame rate drops on mid-range devices, primarily due to over 200 dynamically instantiated sprite game objects that are destroyed and recreated during gameplay. The artist is supplying large, unoptimized texture sheets, and the programmer relies on Unity's default sorting layers and order-in-layer without additional batching strategies. Critically evaluate this development approach, identifying the underlying performance bottlenecks and pipeline inefficiencies. Propose an alternative production strategy that addresses draw call reduction, memory management, and artist–programmer workflow integration, while respecting the constraints of a small team, tight milestone deadlines, and the need to maintain rapid iteration cycles. Justify each decision with specific reference to Unity's 2D rendering pipeline, the Sprite Atlas system, object pooling, and the long-term maintainability of visual assets.

Question 2.

You are the technical lead for a 2D narrative adventure game built in Unity, featuring nonlinear branching dialogue, multiple character sprite sets, and full voice-over integration. The content team—writers, illustrators, and sound designers—work in parallel, frequently updating ScriptableObjects, prefabs, and asset files. The project recently suffered a major production failure: a merge conflict in a binary scene file caused a two-day rollback of narrative content. Your task is to design a production workflow and asset management strategy that prevents such failures while enabling non-programmers to contribute safely. Your proposal must address Unity's prefab variant system, addressable assets for asynchronous loading, scene composition practices (such as multi-scene editing), and version control policies including branching models and binary file locking. Discuss the tradeoffs between asset granularity, loading performance, and content creator autonomy, and justify how your workflow can scale as the team grows from 5 to 15 members.

Question 3.

A small studio is building a 2D physics-based puzzle game in Unity where the player manipulates dozens of dynamic Rigidbody2D objects to trigger chain reactions. During internal playtesting, inconsistent physics simulation across different frame rates caused player complaints about unfair puzzle solutions. The team's initial fix—setting the physics time step to 0.001 seconds and enabling continuous collision detection on all moving objects—severely degraded performance on minimum-spec hardware. Evaluate the team's approach, identifying the conflicting demands of determinism, perceptual fairness, and performance. Propose a rigorous testing and tuning methodology, using Unity's 2D Physics settings, FixedUpdate phasing, interpolation modes, and selective collision detection layers, to achieve an acceptable compromise. In your response, discuss the long-term consequences of your chosen solution for level design iteration, cross-platform deployment, and potential competitive or leaderboard features that rely on replay determinism.

Question 4.

Compare and contrast two distinctly different architectural patterns for a 2D bullet-hell shoot-'em-up developed in Unity:

(a) a traditional GameObject–Component architecture with prefab-based bullets managed by coroutine-driven spawners, and

(b) a data-oriented design leveraging Unity's Entity Component System (ECS) with a custom 2D renderer. For a hypothetical game that must run smoothly on low-end mobile devices while supporting thousands of active projectiles, analyze the tradeoffs between iteration speed, maintainability, memory allocation overhead, draw call efficiency, and team expertise requirements. Defend which architectural approach you would recommend for a four-person team with two junior Unity developers and a strict six-month production schedule. Your justification must address not only runtime performance but also the debugging workflow, the ease of integrating artist-created visual effects, and the long-term extendibility for planned post-launch content updates.